Firjan SENAI

Situação de Aprendizagem

SITUAÇÃO DE APRENDIZAGEM	
Segmento:	Técnico
Unidade (s) Curricular (es):	Desenvolvimento de Sistemas
Curso (s):	Desenvolvimento de Sistemas
Instrutor (a):	Vitor Hugo Almeida de Sousa

ESTRATÉGIA DE APRENDIZAGEM DESAFIADORA

[x] Situação-problema [ ] Estudo do Caso [ ] Projeto [ ] Pesquisa Aplicada

DESCRIÇÃO DA ESTRATÉGIA DE APRENDIZAGEM DESAFIADORA

Contextualização: Abertura de Ordens de Serviço

A equipe de manutenção já consulta setores e equipamentos na API. Agora, precisa registrar os problemas encontrados nas máquinas, identificando o equipamento e o momento da abertura de cada solicitação.

Você acrescentará ao mesmo projeto o cadastro e a consulta de ordens de serviço, utilizando relacionamento entre entidades, DTO, validação e data e hora do sistema.

DESAFIO

Etapa 1 — Conferir o projeto e preparar o Git

- Execute a API e confira GET /equipamentos. Anote o ID de um equipamento existente.
- Utilize o projeto aberturaordensservico e o banco ordens_servico já criados. Consulte o DEV_SIS_11 como referência para relacionamento e DTO.
- Confira git status e registre as alterações anteriores antes de continuar.
- git add . para adicionar as últimas alterações no git.
- git commit -m “mensagem” para registrar no histórico do git

Com a main atualizada, execute:

- git switch main
- git switch -c abertura-os
- git push -u origin abertura-os

Etapa 2 — Criar o Model OrdemServico

- No pacote model, crie OrdemServico.java.
- Acima da classe, use @Entity e @Table(name = "ordem_servico").
- Adicione exatamente estes atributos:
- int id — identificador da ordem; use @Id e @GeneratedValue(strategy = GenerationType.IDENTITY).
- String descricao — descrição do problema apresentado pelo equipamento.
- LocalDateTime dataAbertura — data e hora em que a ordem foi cadastrada. Importe java.time.LocalDateTime.
- Equipamento equipamento — equipamento relacionado à ordem; use @ManyToOne e @JoinColumn(name = "equipamento_id").
- Adicione um construtor vazio, getters e setters.
- Um equipamento poderá ter várias ordens ao longo do tempo; cada ordem aponta para um equipamento. Mantenha o relacionamento somente em OrdemServico.

Etapa 3 — Criar o Repository

- No pacote repository, crie OrdemServicoRepository.java.
- Declare a interface estendendo JpaRepository<OrdemServico, Integer>.

Etapa 4 — Criar o DTO da requisição

No pacote dto, crie OrdemServicoRequest.java.

Declare String descricao com @NotBlank e Integer equipamentoId com @NotNull. Defina mensagens claras no parâmetro message.

Acrescente construtor vazio, getters e setters. O DTO não recebe @Entity.

Receba equipamentoId diretamente no JSON. A dataAbertura será preenchida pelo servidor e não faz parte do DTO.

Exemplo do corpo da requisição:

```
{
  "descricao": "Máquina apresenta falha ao iniciar a costura.",
  "equipamentoId": 1
}
```

Substitua 1 pelo ID real do equipamento. O setor será identificado pelo equipamento associado; não envie setorId nesta requisição.

Etapa 5 — Implementar o Service

- No pacote service, crie OrdemServicoService.java com @Service.
- Receba OrdemServicoRepository e EquipamentoRepository pelo construtor.
- Crie cadastrar recebendo OrdemServicoRequest. Busque o equipamento usando getEquipamentoId().
- Se o equipamento não existir, não salve a ordem. Siga o padrão de Optional vazio e retorno 404 já utilizado no material.
- Se existir, crie OrdemServico, copie a descrição, associe o equipamento encontrado, preencha dataAbertura e salve pelo Repository.

Dica para obter data e hora do sistema

No Service, importe java.time.LocalDateTime e, antes de salvar, utilize:

ordem.setDataAbertura(LocalDateTime.now());

LocalDateTime.now() utiliza a data e a hora do sistema em que a API está executando. O valor é registrado uma vez na abertura; não calcule uma nova data no getter.

Etapa 6 — Criar as rotas no Controller

- No pacote controller, crie OrdemServicoController.java com @RestController e @RequestMapping("/ordens-servico").
- Receba OrdemServicoService pelo construtor.
- POST /ordens-servico: receba @Valid @RequestBody OrdemServicoRequest e encaminhe ao Service. Retorne 201 Created quando cadastrar e 404 Not Found se o equipamento não existir.
- GET /ordens-servico: retorne a lista com 200 OK.
- GET /ordens-servico/{id}: retorne 200 OK quando encontrar a ordem ou 404 Not Found quando o ID não existir.
- Utilize o ValidacaoExceptionHandler já implementado para retornar os campos e as mensagens das validações do DTO.
- Nesta atividade, implemente somente cadastro, listagem e consulta por ID. O encerramento das ordens será uma próxima etapa.

Etapa 7 — Testar e documentar no Postman

- Crie a pasta “Ordens de Serviço” na collection existente.
- Envie POST http://localhost:8080/ordens-servico com o JSON da etapa 4 e um equipamento existente. Espere 201 Created.
- Confira id, descricao, dataAbertura com data e hora e equipamento no retorno. A resposta pode mostrar o objeto equipamento, embora a entrada receba apenas equipamentoId.
- Teste GET /ordens-servico e GET /ordens-servico/{id}; confira a mesma dataAbertura nas consultas.

- Teste descrição vazia ou somente espaços e equipamentoId ausente ou null: espere 400 com as mensagens dos campos.
- Teste equipamentoId inexistente e consulta de ordem inexistente: espere 404. Confirme que os cadastros rejeitados não foram gravados.
- Salve as requisições com suas descrições e resultados esperados.

Etapa 8 — Concluir no Git

Confira se os CRUDs de Setor e Equipamento continuam funcionando.

Registre as rotas e os testes concluídos:

- git add .
- git commit -m "feat: disponibilizar e testar abertura de ordens"
- git push

Após os testes passarem, conclua:

- git switch main
- git merge abertura-os
- git push origin main

• Mantenha a branch abertura-os como registro da atividade.

RESULTADOS E ENTREGAS ESPERADOS
• Entregue o link do mesmo repositório, com os commits da atividade e a branch abertura-os. Exporte a collection com a pasta “Ordens de Serviço”. • Demonstre cadastro válido, listagem, consulta por ID, mensagens de validação e equipamento inexistente. • Confira se a ordem está vinculada ao equipamento correto e se dataAbertura contém a data e a hora registradas pelo servidor. • Explique a diferença entre equipamentoId no DTO e equipamento no Model. Preserve os dados e funcionalidades das atividades anteriores.

Material de Consumo:	Não se aplica
Ferramentas:	IDE; MySQL Workbench; Postman; Git e GitHub; Spring Boot
EPIs:	Não se aplica
EPCs:	Não se aplica
Equipamentos:	Computador com Java/JDK, MySQL e acesso às dependências do projeto
Instrumentos:	Não se aplica